Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Dersin tarihi 11 Kasım 2020 Çarşamba. Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:03:00'te başlar.
  - 00:44:35 ile 01:01:40 arası ara var. Ders yaklaşık 01:14:00'te erken bitti.
- Vize haftası ve vize sayısı (tek vize mi) henüz belli değil.
- Quicksort analizinin tamamlanması (böl ve yönet, divide and conquer)
  - En iyi durum (best case), en kötü durum (worst case), ortalama durum (average case).
  - Pivot seçimi yöntemleri.
- Arama yöntemlerinin özeti
  - Doğrusal arama (sequential search), ikili arama (binary search), ikili arama ağacı (binary search tree).
- Hashing'e giriş
  - Doğrudan adresleme tablosu (direct address table) ve sorunları.
  - Hash fonksiyonu, hash tablosu (hash table), temel işlemler.
  - İyi bir hash fonksiyonunun özellikleri.
  - Bölme yöntemi (division method), çarpma yöntemi (multiplication method).
  - Çakışma (collision) kavramı.

Hocanın Özellikle Vurguladığı Kısımlar

- Quicksort'ta en iyi durum, pivotun her bölmede ortanca (median) değer olmasıdır.
  - Hoca bunu "kural" olarak ekrana yazdı.
- Quicksort'ta en kötü durum sıralı dizidir (pivot ilk eleman seçildiğinde).
  - Soldan sağa tarama $A[1]$'de durur. Sağdan sola tarama dizinin başına kadar gider.
  - Bu yüzden bölme ortada değil, en uçta olur.
- Sıralı veri ile karşılaşılabilecekse ve bellek kısıtı yoksa Mergesort tercih edilebilir.
  - Mergesort kararlıdır (stable). Quicksort'un en kötü durumu $\Theta(n^2)$'dir.
- Quicksort'un performansını doğrudan etkileyen şey pivot seçimidir.
  - İlk eleman, son eleman, ortanca ya da rastgele seçilebilir.
- İkili arama yalnız sıralı veri üzerinde çalışır.
- İkili arama ağacı dengeli değilse en kötü durum $O(n)$ olur.
  - Örnek: elemanlar sıralı (ya da ters sıralı) eklenirse ağaç bir tarafa "yıkılır".
- Hash tablosunda temel işlemler arama (search), ekleme (insert) ve silme (delete) işlemleridir.
- Hash fonksiyonu seçerken üç şeye dikkat edilir:
  - Kolay hesaplanabilmesi (easy to compute), yani az matematik işlemi.
  - Tutarlılık (consistency).
  - Düzgün adres üretimi (uniform address generation).
- Bölme yönteminde m olarak asal sayı seçilir. 2'nin kuvvetleri ve 10 gibi değerlerden kaçınılır.
- Hoca derse katılım konusunda uyarı yaptı: Derste söz alınmadan ders ilerlemeyecek.

Kısa Tekrar Notları

- Quicksort en iyi durum: $C_{best}(n) = 2C_{best}(n/2) + n$, $C_{best}(1) = 0$.

  - Master teoremi: $a = 2$, $b = 2$, $d = 1 \Rightarrow \Theta(n \log n)$.
  - Geriye doğru yerine koyma ile de $n \log_2 n$ bulunur.
- Quicksort en kötü durum: $C_{worst}(n) = (n+1) + n + \cdots + 3 = \frac{(n+1)(n+2)}{2} - 3 \in \Theta(n^2)$.
- Quicksort ortalama durum: $C_{avg}(n) \approx 2n \ln n \approx 1.39\, n \log_2 n$.
  - En iyi duruma göre yaklaşık %39 daha fazla karşılaştırma.
- Ödevdeki en yakın nokta çifti (minimum distance) sorusunun verimli çözümü: önce sıralama, sonra tarama.
  - $n \log n + n$, yani $O(n \log n)$.
- Arama karmaşıklıkları (en kötü durum):
  - Doğrusal arama: $O(n)$.
  - İkili arama: $O(\log n)$.
  - İkili arama ağacı: dengeli ise $O(\log n)$, dengesiz ise $O(n)$.
  - Hashing: ortalama $O(1)$.
- Doğrudan adresleme tablosu: anahtarın kendisi dizi indisidir. Anahtar uzayı büyüdüğünde bellek israfı olur.
- Bölme yöntemi: $h(k) = k \bmod m$. Örnek: $m = 11$, $k = 25 \Rightarrow h(25) = 3$.
- Çakışma (collision): farklı anahtarların aynı adrese düşmesi.
  - Çözümler: açık adresleme (open addressing) ve ayrık zincirleme (separate chaining). Sonraki derste işlenecek.

Detaylı Açıklamalar

Quicksort'ta en iyi durum

Pivot her seferinde ortanca değer olursa dizi tam ortadan ikiye bölünür. İki yarı ayrı ayrı çözülür. Bölme işlemi (partition) n karşılaştırma yapar.

$$C_{best}(n) = 2C_{best}(n/2) + n, \quad C_{best}(1) = 0$$

Master teoreminde $a = 2$, $b = 2$, $d = 1$ olduğu için $a = b^d$ durumu oluşur. Sonuç $C_{best}(n) \in \Theta(n \log n)$'dir. $n = 2^k$ alınıp geriye doğru yerine koyma yapılırsa da $n \log_2 n$ bulunur.

Quicksort'ta en kötü durum

Dizi sıralı olsun: 1, 2, 3, 4, 5, 6, 7. Pivot ilk eleman (1) seçilir.

- Soldan sağa tarama (left-to-right scan) pivottan büyük ilk elemanı arar. Hemen $A[1] = 2$'de durur.
- Sağdan sola tarama (right-to-left scan) pivottan küçük eleman arar. Hiç bulamaz, $A[0]$'a kadar gider.
- İndisler ortada buluşmaz. Bölme en uçta olur: bir tarafta 0 eleman, diğer tarafta $n - 1$ eleman kalır.

Bir turdaki karşılaştırma sayısı $n+1$'dir. Sonraki turlar n, $n-1$, ... diye azalır. En son iki eleman kaldığında 3 karşılaştırma yapılır: iki tarama birer karşılaştırma yapar, bir karşılaştırma da indislerin birbirini geçmesi içindir.

$$C_{worst}(n) = (n+1) + n + \cdots + 3 = \frac{(n+1)(n+2)}{2} - 3 \in \Theta(n^2)$$

$n = 7$ için toplam 33 karşılaştırma yapılır.

Quicksort'ta ortalama durum

Varsayım: Dizi rastgele sıralıdır. Bölme noktası s ($0 \le s \le n-1$) her konumda eşit olasılıkla ($1/n$) oluşur.

$$C_{avg}(n) = \frac{1}{n}\sum_{s=0}^{n-1}\left[(n+1) + C_{avg}(s) + C_{avg}(n-1-s)\right], \quad C_{avg}(0) = C_{avg}(1) = 0$$

Çözümü: $C_{avg}(n) \approx 2n \ln n \approx 1.39\, n \log_2 n$.

Arama yöntemleri

- Doğrusal arama: sıralı ve sıralı olmayan dizide çalışır. En kötü durum $O(n)$.
- İkili arama: dizi sıralı olmalıdır. En kötü durum $O(\log n)$.
- İkili arama ağacı (BST): her düğümün solunda küçük, sağında büyük anahtarlar bulunur.
  - Elemanlar sıralı eklenirse ağaç tek bir dala dönüşür. Arama $O(n)$ olur.
- Heap'te arama verimli değildir. Heap yalnız en büyüğü (max-heap) ya da en küçüğü (min-heap) hızlı verir.

Hashing

Bir anahtar (key) hash fonksiyonuna verilir. Fonksiyon bir adres üretir. Değer (value) bu adrese yazılır. Bu alana hash tablosu denir.

- Doğrudan adresleme tablosu: Her olası anahtar için bir dizi gözü ayrılır. Örneğin $A[0]$, $A[1509]$, $A[12782]$.
  - Sorun 1: Anahtar uzayı büyükse (örneğin 0-1.000.000) dizi çok büyük olur. Az sayıda anahtar saklanırken gözlerin çoğu boş kalır.
  - Sorun 2: Anahtar sayı değilse (örneğin string) önce tam sayıya çevirmek gerekir.
- Hash tablosu, büyük anahtar uzayını küçük bir tabloya eşler (map). Bunun bedeli çakışmadır.
- Diziye eleman eklemek maliyetlidir. Bağlı listede ekleme ucuzdur ama arama pahalıdır. Hash tablosu arama, ekleme ve silmeyi ortalamada $O(1)$ yapar.

Bölme yöntemi: $h(k) = k \bmod m$. m genelde tablo boyutudur. $m = 10$ seçilirse 10, 40, 50, 100 anahtarlarının hepsi 0 adresine düşer. Bu düzgün bir dağılım değildir. Bu yüzden m asal seçilir.

Çarpma yöntemi: Ders 13'te ayrıntılı işlendi.

Ders sonrası notlar

- **Quicksort ortalama sabiti:** Hoca "$1.38\, n \log_2 n$, yaklaşık %38 daha fazla" dedi (00:30:22). $2 \ln 2 \approx 1.386$ olduğu için kaynaklarda (Levitin) değer 1.39 ve "yaklaşık %39" olarak verilir.
- **En kötü durum yalnız sıralı dizi değildir:** Pivot ilk eleman seçildiğinde, kesin azalan (ters sıralı) dizi de en kötü durumdur. Derste bir öğrenci "pivot en büyük olsaydı?" diye sordu, bu soru cevaplanmadı. Pivot en büyük eleman olursa da bölme en uçta olur ve karmaşıklık yine $\Theta(n^2)$'dir.
- **İkili arama ağacının gösterimi:** Bir öğrenci BST'yi "$2i$ ve $2i+1$ indisleriyle" anlattı. Bu, tam ikili ağacın (örneğin heap) dizi gösterimidir. BST genellikle düğüm ve işaretçilerle (pointer) tutulur. BST'yi tanımlayan şey indisler değil, "sol alt ağaç küçük, sağ alt ağaç büyük" sıralama özelliğidir.
- **Hash ve kriptografi:** Hoca bölme yönteminin "tek yönlü (one-way)" olduğunu ve bu yüzden kriptografide kullanıldığını söyledi. Hash tablolarında kullanılan k mod m gibi fonksiyonlar kriptografik değildir. Kriptografide SHA-256 gibi özel tasarlanmış, çakışmaya dayanıklı (collision-resistant) fonksiyonlar kullanılır. k mod m'den bir anahtar bulmak kolaydır.
- **Hashing karmaşıklığı:** $O(1)$ ortalama durumdur. Kötü bir hash fonksiyonunda bütün anahtarlar aynı adrese düşerse en kötü durum $O(n)$ olur.
- Altyazıda "Best Gaze", "Divide and Concur", "Core Partition", "collusion" diye geçen terimlerin doğruları: best case, divide and conquer, Hoare partition, collision.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.